

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

// Create Node

struct Node {

    int key;

    struct Node *left;

    struct Node *right;

    int height;

};

// Function prototypes

int max(int a, int b);

int height(struct Node *N);

struct Node *newNode(int key);

struct Node *rightRotate(struct Node *y);

struct Node *leftRotate(struct Node *x);

int getBalance(struct Node *N);

struct Node *insertNode(struct Node *node, int key);

struct Node *minValueNode(struct Node *node);

struct Node *deleteNode(struct Node *root, int key);

void printPreOrder(struct Node *root);

// Utility function to get the maximum of two integers

int max(int a, int b) {

    return (a > b) ? a : b;

}

// Utility function to get the height of the tree
```

```

int height(struct Node *N) {    if (N == NULL)

    return 0;

    return N->height;

}

// Utility function to create a new node with given key
struct Node *newNode(int key) {

    struct Node *node = (struct Node *) malloc(sizeof(struct Node));

    node->key = key;

    node->left = NULL;

    node->right = NULL;

    node->height = 1;

// New node is initially added at leaf

return node;

}

// Utility function to perform right rotation
struct Node *rightRotate(struct Node *y) {

struct Node *x = y->left;

struct Node *T2 = x->right;

// Perform rotation

    x->right = y;

y->left = T2;

// Update heights

    y->height = max(height(y->left), height(y->right)) + 1;

    x->height = max(height(x->left), height(x->right)) + 1;

// Return new root

return x;

```

```

    }

    // Utility function to perform left rotation
    struct Node *leftRotate(struct Node *x) {
        struct Node *y = x->right;
        struct Node *T2 = y->left;

        // Perform rotation
        y->left = x;
        x->right = T2;

        // Update heights
        x->height = max(height(x->left), height(x->right)) + 1;
        y->height = max(height(y->left), height(y->right)) + 1;

        // Return new root
        return y;
    }

    // Utility function to get the balance factor of a node
    int getBalance(struct Node *N) {
        if (N == NULL)    return 0;

        return height(N->left) - height(N->right);
    }

    // Recursive function to insert a key in the subtree rooted with node
    struct Node *insertNode(struct Node *node, int key) {

        // Perform the normal BST insertion
        if (node == NULL)
            return newNode(key);

        if (key < node->key)
            node->left = insertNode(node->left, key);

```

```

else if (key > node->key)

    node->right = insertNode(node->right, key);

else

// Equal keys are not allowed in BST

    return node;

// Update the height of this ancestor node

node->height = 1 + max(height(node->left), height(node->right));

// Get the balance factor of this ancestor node to check whether this node became unbalanced

int balance = getBalance(node);

// If this node becomes unbalanced, then there are 4 cases

// Left Left Case

if (balance > 1 && key < node->left->key)

    return rightRotate(node);

// Right Right Case

if (balance < -1 && key > node->right->key)

    return leftRotate(node);

// Left Right Case

if (balance > 1 && key > node->left->key) {

    node->left = leftRotate(node->left);

    return rightRotate(node);

}

// Right Left Case

if (balance < -1 && key < node->right->key) {

    node->right = rightRotate(node->right);

    return leftRotate(node);

}

```

```

    // Return the (unchanged) node pointer
    return node;
}

// Function to find the node with minimum key value in a subtree rooted with node
struct Node *minValueNode(struct Node *node) {
    struct Node *current = node;

    // Loop down to find the leftmost leaf
    while (current->left != NULL)
        current = current->left;

    return current;
}

// Recursive function to delete a node with given key from the subtree rooted with node
struct Node *deleteNode(struct Node *root, int key) {
    // STEP 1: PERFORM STANDARD BST DELETE

    if (root == NULL)
        return root;

    // If the key to be deleted is smaller than the root's key, then it lies in left subtree
    if (key < root->key)
        root->left = deleteNode(root->left, key);

    // If the key to be deleted is greater than the root's key, then it lies in right subtree
    else if (key > root->key)
        root->right = deleteNode(root->right, key);

    // If key is same as root's key, then this is the node to be deleted
    else {
        // Node with only one child or no child
        if ((root->left == NULL) || (root->right == NULL)) {

```

```

    struct Node *temp = root->left ? root->left : root->right;

    // No child case
    if (temp == NULL) {
        temp = root;
        root = NULL;
    } else
// One child case
        *root = *temp; // Copy the contents of the non-empty child
        free(temp);
    }

    else {          // Node with two children: Get the inorder successor (smallest in the right subtree)
        struct Node *temp = minValueNode(root->right);

        // Copy the inorder successor's data to this node
        root->key = temp->key;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->key);
    }
}

// If the tree had only one node then return
if (root == NULL)
    return root;

// STEP 2: UPDATE HEIGHT OF THE CURRENT NODE
root->height = 1 + max(height(root->left), height(root->right));

// STEP 3: GET THE BALANCE FACTOR OF THIS NODE (to check whether this node became
unbalanced)
int balance = getBalance(root);

// If this node becomes unbalanced, then there are 4 cases

```

```

// Left Left Case

if (balance > 1 && getBalance(root->left) >= 0)

    return rightRotate(root);

// Left Right Case

if (balance > 1 && getBalance(root->left) < 0) {

    root->left = leftRotate(root->left);

    return rightRotate(root);

}

// Right Right Case

if (balance < -1 && getBalance(root->right) <= 0)

    return leftRotate(root);

// Right Left Case

if (balance < -1 && getBalance(root->right) > 0) {

    root->right = rightRotate(root->right);

    return leftRotate(root);

}

return root;

}

// A utility function to print preorder traversal of the tree

void printPreOrder(struct Node *root) {

    if (root != NULL) {

        printf("%d ", root->key);

        printPreOrder(root->left);

        printPreOrder(root->right);

    }

}

```

```

// Main function

int main() {

    struct Node *root = NULL;

    // Insert nodes

    root = insertNode(root, 9);
    root = insertNode(root, 5);
    root = insertNode(root, 10);
    root = insertNode(root, 0);
    root = insertNode(root, 6);
    root = insertNode(root, 11);
    root = insertNode(root, -1);
    root = insertNode(root, 1);
    root = insertNode(root, 2);

    printf("Preorder traversal of the constructed AVL tree is \n");
    printPreOrder(root);

    // Delete nodes

    root = deleteNode(root, 10);

    printf("\n\nPreorder traversal after deletion of 10 \n");
    printPreOrder(root);

    return 0;
}

```

OUTPUT:

Preorder traversal of the constructed AVL tree is

9 1 0 -1 5 2 6 10 11

Preorder traversal after deletion of 10

1 0 -1 9 5 2 6 11